# Robotic Manipulation Workflow

## MuJoCo Simulation, Teleoperation, Data Recording, Training, and Policy Deployment

Technical handover for the LIRMM DEXTER team


| | |
|---|---|
| Prepared from: | the internship implementation by Luca Vogelgesang |
| Scientific supervision: | Chao Liu |
| Team: | LIRMM DEXTER |
| Version: | 1.5 |
| Date: | 4 September 2026 |

# Contents

Chapter 1

# MuJoCo Fundamentals

This guide follows the shortest useful path through the project: open and validate the MuJoCo task, control it with the 3D Systems Touch, record and inspect demonstrations, train a policy, and execute the checkpoint. It assumes basic knowledge of Python, ROS 2, and robotics. Installation commands are kept in `README_ENV.md`; the complete Touch installation is reproduced in appendix A.

MuJoCo reads an XML description of a mechanical system and numerically advances its state through time. In this project it provides the UR10, microwave scene, contacts, cameras, robot state, and control interface needed to test the full learning workflow without moving physical hardware.

## 1.1 Use the repository guides together

The documentation is divided by execution context:

| File | Use |
|---|---|
| `README_ENV.md` | Install the local Conda, ROS 2, MuJoCo, and Touch environment. |
| `README_CODE.md` | Find maintained launch commands, controls, data tools, and program locations. |
| `README_SERVER.md` | Transfer a dataset, enter the training container, launch training, and recover checkpoints. |

This PDF explains the order of operations and the checks between them. When a copyable command differs from the source code, the corresponding README is the authoritative reference.

## 1.2 Activate the local environment

From a new terminal:

```
cd "$HOME/Stage_Lirmm/Diffusion-model-isaacsim"
conda activate microwave_dp
python --version
python -c "import mujoco, numpy, cv2; print('Local environment: OK')"
```

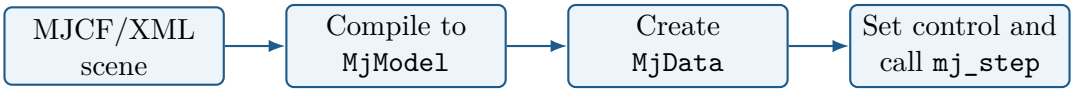
If `microwave_dp` does not exist, stop and follow `README_ENV.md`. Do not install packages one by one into an unrelated environment; the resulting versions may no longer match the project.

**Local pre-flight**

The repository is complete, `microwave_dp` activates without an error, and Python can import MuJoCo, OpenCV, NumPy, Pinocchio, ROS 2, and the local `diffusion_policy` package.

## 1.3 What MuJoCo Executes

MuJoCo is a physics engine for articulated systems, contacts, and rigid-body dynamics. A typical program has four steps:




The distinction between model and data is fundamental:

**`mujoco.MjModel`:** the compiled, mostly fixed description of bodies, joints, masses, geometry, cameras, actuators, and limits.

**`mujoco.MjData`:** the changing state of one simulation, including joint position, velocity, control input, contact information, body pose, and simulation time.

**`mujoco.mj_step`:** advances dynamics by one model time step.

**`mujoco.mj_forward`:** recomputes derived positions and velocities after state was changed directly, without advancing time.

An XML edit requires the model to be reloaded. In contrast, moving a free object at reset normally modifies `MjData` and then calls `mj_forward`.

## 1.4 Run the Minimal Example

The repository contains a small example independent of the microwave task:

- `dp_mujoco/examples/minimal_scene.xml`;
- `dp_mujoco/examples/mujoco_basics.py`.

It contains a floor, light, one actuated hinge, a free cube, a site, and a camera. Run it without graphics first:

```
python dp_mujoco/examples/mujoco_basics.py
```

The output reports the simulation time step and the dimensions `nq`, `nv`, and `nu`, then prints the changing joint position and control target. Open the same example interactively with:

```
python dp_mujoco/examples/mujoco_basics.py --viewer --duration 10
```

The blue arm follows a sinusoidal target, the green cube falls under gravity, and the red site follows the end of the arm. This one scene demonstrates actuation, free-body dynamics, contact, and a kinematic reference frame.

## 1.5 Read a Small MJCF Scene

MJCF describes a hierarchy. Child bodies move with their parent, while joints define the motion allowed between them. The important elements are:

| Element | Purpose |
|---|---|
| `body` | A rigid frame that can contain joints, geometry, sites, and child bodies. |
| `joint` | One degree of freedom such as a hinge or slide. |
| `freejoint` | Six free degrees of freedom, represented by 3D position and a quaternion. |
| `geom` | Visual and/or collision geometry with size, mass, friction, and material. |
| `site` | A massless reference frame used for a TCP, sensor location, or target. |
| `camera` | A fixed or body-mounted viewpoint used by the viewer or renderer. |
| `actuator` | Converts a value in `data.ctrl` into force, velocity, or position control. |

The core of the tutorial scene is deliberately small:

```
<body name="arm" pos="0 0 0.08">
  <joint name="arm_hinge" type="hinge" axis="0 1 0"
         range="-1.57 1.57"/>
  <geom type="capsule" fromto="0 0 0 0 0 0.35"
        size="0.025" mass="0.5"/>
  <site name="tool_site" pos="0 0 0.35" size="0.015"/>
</body>

<actuator>
  <position name="arm_position" joint="arm_hinge" kp="20"/>
</actuator>
```

The actuator type defines the meaning of `data.ctrl`. Here the value is a desired hinge angle. A motor actuator would interpret it differently, so never assume that `ctrl` always contains joint positions.

MuJoCo uses half-extents for a box `geom`: a box with `size="0.040.040.04"` measures $8 \times 8 \times 8$ cm. Incorrectly treating these values as full dimensions is a common source of collision errors.

## 1.6 Understand the Python Simulation Loop

The essential Python loop is:

```
model = mujoco.MjModel.from_xml_path("minimal_scene.xml")
data = mujoco.MjData(model)

actuator_id = mujoco.mj_name2id(
    model, mujoco.mjtObj.mjOBJ_ACTUATOR, "arm_position"
)

while data.time < 4.0:
    data.ctrl[actuator_id] = target_angle
    mujoco.mj_step(model, data)
```

The most frequently inspected arrays are:

**`data.qpos`:** generalized positions. A hinge uses one value; a free joint uses seven values: three for position and four for orientation.

**`data.qvel`:** generalized velocities. A free joint uses six values, so `nq` and `nv` need not be equal.

**`data.ctrl`:** one input per actuator; its length is `model.nu`.

**`data.site_xpos/site_xmat`:** world position and orientation of sites after forward kinematics or a simulation step.

Use names instead of hard-coded indices when possible. The project obtains the TCP site in the same way:

```
site_id = mujoco.mj_name2id(
    model, mujoco.mjtObj.mjOBJ_SITE, "grasp_site"
)
tcp_position = data.site_xpos[site_id].copy()
```

## 1.7 Open the Project Scene

The active task scene is:

`dp_mujoco/models/universal_robots_ur10e/scene_microwave_camera.xml`

The scene includes `ur10e_with_gripper.xml`; this keeps robot links, joints, gripper, actuators, wrist camera, and `grasp_site` separate from the microwave, table, objects, target boxes, lighting, and global camera. The Python files around the XML are:

| Path | Responsibility |
|---|---|
| `dp_mujoco/env/mujoco_env.py` | Loads the scene and exposes state, rendering, reset, and commands. |
| `dp_mujoco/env/scene_utils.py` | Changes object pose during teleoperation resets. |
| `dp_mujoco/utils/scene_utils.py` | Applies the matching reset distribution during policy execution. |
| `dp_mujoco/teleop/test_UR10e_touch.py` | Runs simulated teleoperation and recording. |
| `dp_mujoco/policy_exec/run_diffusion_mujoco.py` | Executes a trained checkpoint in MuJoCo. |

Open the XML without ROS or the Touch:

```
python -m mujoco.viewer \
  --mjcf=dp_mujoco/models/universal_robots_ur10e/scene_microwave_camera.xml
```

Inspect the body hierarchy, joint limits, contact geometry, and initial object poses. The rendered mesh may look correct even when its collision geometry is wrong, so enable contact or geometry visualization when debugging.

## 1.8 Separate Visual and Collision Geometry

A detailed mesh is useful for appearance but often unnecessarily difficult for contact. In the project scene, the microwave OBJ mesh has collision disabled and simple transparent boxes represent its floor, walls, and back. This makes contact faster and easier to inspect.

When adding an object, begin with a primitive:

```
<body name="practice_cube" pos="0.60 0.00 0.48">
  <freejoint/>
  <geom type="box" size="0.025 0.025 0.025"
        mass="0.10" rgba="0.15 0.55 0.90 1"/>
</body>
```

Reload the XML and let the object settle. If it falls through, jumps, or becomes stuck, check units, initial overlap, collision dimensions, mass, friction, and contact settings before modifying the robot controller.

## 1.9 Render the Policy Cameras

The project uses `top_table` as the global camera and `wrist_cam` as the eye-in-hand camera. A renderer must be updated from the latest `MjData` before an image is read:

```
renderer = mujoco.Renderer(model, height=480, width=640)
renderer.update_scene(data, camera="top_table")
rgb_image = renderer.render()
```

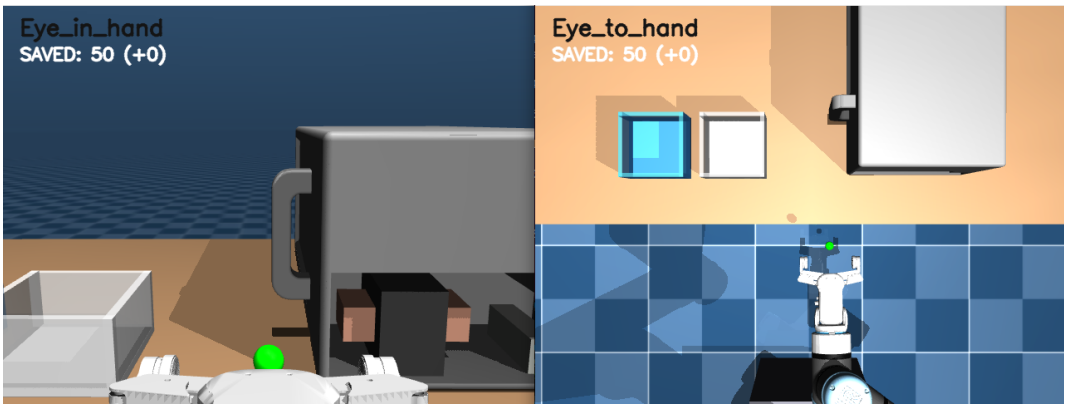



Figure 1.1: Project camera observations: wrist view on the left and global view on the right.

The teleoperation display uses $640 \times 480$ images, while its recorder resizes them to $84 \times 84$ for the simulation dataset. Camera name, ordering, orientation, and final image size must remain identical during policy execution.

## 1.10 Reset and Randomize an Object

A free joint stores object position followed by a quaternion in `data.qpos`. The project reset utility finds the free-joint address, writes the new pose, clears its six velocity values, and calls `mj_forward`:

```
qpos_adr = model.jnt_qposadr[joint_id]
qvel_adr = model.jnt_dofadr[joint_id]

data.qpos[qpos_adr:qpos_adr + 3] = position
data.qpos[qpos_adr + 3:qpos_adr + 7] = quat_wxyz
data.qvel[qvel_adr:qvel_adr + 6] = 0.0
mujoco.mj_forward(model, data)
```

The reset distribution used for collection and policy execution must match. The current repository has one randomization module for each path; update and test both when the pose range changes. Run many resets before recording and confirm that every object pose is collision-free, visible, reachable, and part of the intended task.

**MuJoCo environment ready**

The minimal example runs, the project XML opens, the robot and free objects behave correctly under gravity and contact, both cameras render useful views, and repeated reset samples remain valid. Only then connect the Touch and test the complete teleoperation loop.

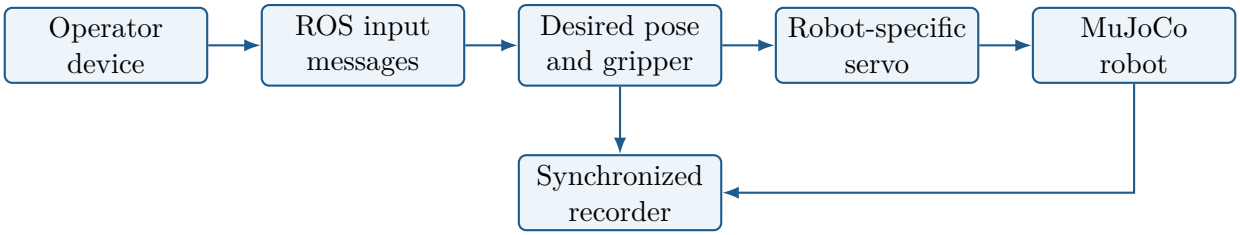
Chapter 2

# Teleoperate in Simulation

The 3D Systems Touch is used to generate Cartesian targets for the simulated robot. Install and validate the device before starting this chapter; the full procedure is in appendix A. The Touch must be plugged in, and no other OpenHaptics program may own it while the teleoperation launcher is active.

## 2.1 The Project Teleoperation Loop

A practical teleoperation system separates device input, target generation, low-level control, and recording:




The recorder observes the same simulation while the operator controls it. This keeps the teleoperation loop and the saved demonstrations synchronized.

## 2.2 How the Touch Motion Is Mapped

The physical coordinates of a desktop device generally have no useful absolute relationship with the robot base. Use differential motion:

$$\Delta p_{robot} = s_p M_p \left( p_t^{device} - p_{t-1}^{device} \right), \tag{2.1}$$

where $M_p$ maps axes and $s_p$ sets workspace scale. The first valid device pose becomes a reference and produces no robot movement. This prevents the initial jump that would occur if the device pose were interpreted as an absolute robot target.

Test one translation and one rotation axis at a time. If an axis is reversed, correct the mapping in the code instead of compensating during demonstrations.

For Cartesian velocity control, a common structure is:

$$v_{des} = \begin{bmatrix} K_p e_p & K_r e_r \end{bmatrix}^T, \qquad \dot{q} = J^{\#}(q) v_{des}, \tag{2.2}$$

where $e_p$ and $e_r$ are pose errors and $J^{\#}$ is a damped Jacobian pseudoinverse. The controller then applies joint-velocity and safety limits.

Target smoothing can reduce abrupt hand motion:

$$x_f(t) = (1 - \alpha)x_f(t - 1) + \alpha x_{raw}(t). \tag{2.3}$$

A large $\alpha$ follows the hand more closely; a small value smooths motion but adds lag. Change workspace scale, target-speed limit, smoothing, and controller gain one at a time. The robot must be responsive enough for small corrections before any data are recorded.

## 2.3 Launch the Complete Simulation Loop

Close standalone Touch diagnostics first, then run:

```
cd "$HOME/Stage_Lirmm/Diffusion-model-isaacsim"
conda activate microwave_dp
./launch_all.sh
```

The wrapper checks required files and Python modules, starts the ROS 2 Touch node, waits for `/touch/pose`, and then opens the MuJoCo teleoperation program. Both processes stop when `Ctrl+C` is pressed in this terminal. Touch logs are written to `/tmp/diffusion_policy_touch_driver.log` by default.

Before recording, move the stylus slowly and test all axes. Confirm that:

- the first received Touch pose does not make the robot jump;
- translations and rotations follow the expected directions;
- the gripper buttons produce the expected simulated command;
- the robot remains stable when the Touch is held still;
- the wrist and global views remain live throughout the motion.

## 2.4 Fix Responsiveness Before Recording

If small corrections are difficult, do not compensate by making large hand motions. Check the newest ROS target is consumed, then inspect position scale, target filtering, servo gains, joint-velocity limits, and simulation rate. A slow controller records delayed corrective motion that the future policy will learn.

**Common Touch launch failures**

If `/touch/pose` never appears, run the vendor diagnostic and ROS topic test from appendix A. If the device is reported as busy, close every other OpenHaptics example or Touch ROS node. If the first workspace build fails, verify `OPENHAPTICS_ROOT` before rebuilding.

**Teleoperation ready**

The simulated robot follows every mapped axis without jumps, lag is low enough for precise corrections, the gripper behaves consistently, and both policy camera views remain live during a complete manual task.

Chapter 3

# Record and Check Demonstrations

The MuJoCo teleoperation program records complete task attempts as episodes in a Zarr dataset. Recording and control occur in the same program, so the saved images, measured state, and demonstrated target share one timeline.

## 3.1 Record Episodes

Start `./launch_all.sh` as described in chapter 2. The MuJoCo window uses these controls:

| Key | Action |
|---|---|
| `Space` | Start recording; press again to stop and save the episode. |
| `Backspace` | Cancel the current episode without saving it. |
| `R` | Reset the robot and randomize the scene. |
| `Esc` | Quit. |

The recorder creates a timestamped dataset under `data/datasets/` and samples at 10 Hz. Practice before recording. Start just before useful motion, finish after task completion, and cancel failed, interrupted, or corrupted attempts rather than teaching them as intended behavior.

## 3.2 Define one synchronized sample

A visuomotor sample normally contains:

$$(o_t, a_t) = (\text{images}_t, \text{robot state}_t, \text{demonstrated command}_t)\,. \tag{3.1}$$

An episode is the ordered sequence from one start/stop interval. Episode boundaries must remain intact because a training window must never cross from the end of one attempt into the reset of the next.

## 3.3 Define and document the schema

The exact keys are project-specific. For example, the internship recorder stored the following Zarr arrays at 10 Hz:

Table 3.1: MuJoCo dataset interface used by the project recorder.

| Key | Per-step shape | Meaning |
|---|---|---|
| `agentview_image` | $84 \times 84 \times 3$ | Global RGB image. |
| `robot0_eye_in_hand_image` | $84 \times 84 \times 3$ | Wrist RGB image. |
| `robot0_eef_pos` | 3 | Measured end-effector position. |
| `robot0_eef_quat` | 4 | Measured end-effector quaternion. |
| `robot0_gripper_qpos` | 1 | Measured gripper opening/state. |
| `action` | 8 | Target pose and gripper command. |

The simulation action is

$$a_t = [x, y, z, q_x, q_y, q_z, q_w, g]. \tag{3.2}$$

The quaternion stored in the action is `xyzw`; the measured `robot0_eef_quat` observation is `wxyz`. This project-specific difference is handled by the current recorder and runner, but must not be silently changed or mixed with real-robot datasets.

## 3.4 Inspect structure, content, and coverage

Validate every new dataset before sending it to the training server.

First confirm episode count, equal array lengths, action dimension, image shapes, and valid Zarr structure:

```
python scripts/check_zarr.py data/datasets/<dataset>.zarr
```

Then replay every episode and inspect the robot trajectory:

```
python scripts/replay_zarr.py data/datasets/<dataset>.zarr --scale 2
python scripts/traj_zarr.py data/datasets/<dataset>.zarr
```

Look for frozen or repeated images, long idle intervals, abrupt target jumps, unintended corrective loops, incorrect gripper transitions, and actions that cannot be explained from the recorded observations. Also verify that object positions and orientations cover the range expected during policy execution.

**Demonstration quality**

A successful episode is not automatically a good demonstration. Keep motions deliberate and consistent, avoid hiding the object at critical moments, and cancel an episode when the operator makes a correction that should not be imitated. Record three episodes first and inspect all of them before continuing with a long session.

Keep the original dataset immutable after training begins. If episodes must be removed or new demonstrations added, create a new named version so that each checkpoint can still be linked to the data that produced it.

**Interface freeze**

The structure checker passes, every accepted episode has been replayed, camera frames remain live, actions match operator intent, and the demonstrated pose range matches the task that will be tested.

Chapter 4

# Train and Deploy the Policy

Training is the only stage normally run on the GPU server. The exact Docker, transfer, and training commands are maintained in `README_SERVER.md`; use that guide rather than reconstructing an old terminal history.

## 4.1 Keep the Four Execution Contexts Distinct

| Context | Typical work |
|---|---|
| Local experiment PC | Collect, replay, and validate the Zarr dataset. |
| GPU server host | Store the transferred project and start or enter Docker. |
| Docker container | Activate `robodiff` and run training from `/workspace`. |
| Local deployment PC | Load and execute the copied checkpoint in MuJoCo or on the robot. |

The path `/workspace` exists only inside the container. Commands that fail with a missing `/workspace` on the local PC are being run in the wrong context.

## 4.2 Minimal Training Sequence

1. Check the final local dataset and stop editing it.
2. Transfer that dataset, the matching task/training YAML files, and the source required by the selected model.
3. Enter the persistent Docker container and activate `robodiff`.
4. Print the dataset path inside Docker and pass it explicitly as `task.dataset_path`.
5. Launch the matching training configuration inside a `tmux` session and monitor GPU use with `nvidia-smi`.
6. Copy several checkpoint candidates back to `data/checkpoints/` on the local PC.

For a MuJoCo dataset recorded by this guide, the matching configurations are `train_e_waste` and `e_waste_image`. After following the container and configuration-copy steps in `README_SERVER.md`, launch this example from inside Docker:

```
cd /workspace
source /opt/conda/etc/profile.d/conda.sh
conda activate robodiff

DATASET_NAME="<simulation_dataset>.zarr"

python /workspace/diffusion_policy/train.py \
  --config-name=train_e_waste \
  task=e_waste_image \
```

```
  task.dataset_path="/workspace/data/datasets/$DATASET_NAME" \
  training.resume=False \
  training.device=cuda:0 \
  logging.mode=disabled \
  logging.resume=False
```

For another task or model, replace both configuration names together. The task YAML must describe exactly the recorded keys, image sizes, state dimensions, action dimensions, and horizons. A checkpoint trained with a real dataset configuration must not be tested with the simulation interface, or the reverse.

Select a few checkpoints around the best validation region and test them under the same conditions. Validation loss is useful for identifying overfitting, but the final choice must also produce stable closed-loop behavior.

**Checkpoint ready**

The checkpoint loads on the local PC, its dataset and resolved configuration are known, and its observation/action interface matches the runner that will execute it.

## 4.3 Deploy the Checkpoint in MuJoCo

Always execute a simulation checkpoint in MuJoCo before considering physical deployment. Copy it to `data/checkpoints/`, activate the local environment, and run:

```
cd "$HOME/Stage_Lirmm/Diffusion-model-isaacsim"
conda activate microwave_dp

python -m dp_mujoco.policy_exec.run_diffusion_mujoco \
  --checkpoint data/checkpoints/<simulation_model>.ckpt \
  --device cpu \
  --policy_hz 10 \
  --exec_horizon 8 \
  --debug_timing
```

The runner builds the same camera/state observations as training, asks the policy for a future action sequence, executes a prefix, and replans. Use `--help` to inspect model-path, camera, controller, smoothing, timing, and trajectory-recording options.

Before judging model quality, verify:

- the correct MuJoCo scene and checkpoint are loaded;
- observation keys, image order, and image size match training;
- the action frame, quaternion order, scale, and gripper convention match;
- the reset distribution is the same one used during collection;
- inference time is compatible with the chosen action execution rate.

If the robot immediately moves toward an implausible pose, stop the runner and check the interface before changing gains. A wrong quaternion convention or normalizer can look like a controller problem.

## 4.4 Optional Physical-Robot Deployment

The physical runner is intentionally separate from MuJoCo. It must recreate the training observation interface using real cameras and robot state, then decode actions through a robot-specific controller. The maintained UR10 wrapper is:

`ur10_real_robot/run_diffusion_real.sh`

Follow the staged commands in `README_CODE.md`: first load the model with fake inputs, then use real observations with motion disabled, and only then enable low-speed motion. Compare live camera images with the dataset before moving the robot.

On the project laptop, inference can be slower than the 10 Hz action execution rate. The real runner can therefore execute a buffered plan while the next plan is computed asynchronously. This reduces pauses but the new plan is based on an older observation; inference latency and robot speed must still be measured and kept compatible.

**Physical execution**

Never execute an unknown checkpoint first with motion enabled. Keep independent position, orientation, velocity, and timing stops; begin with a low teach-pendant slider; clear the workspace; and keep an operator beside the emergency stop. A safety stop is evidence to diagnose, not a threshold to raise automatically.

## 4.5 Short Troubleshooting Order

When execution fails, check the pipeline in the same order in which it was built:

| Symptom | First check |
|---|---|
| Scene or reset is wrong | Open the XML alone; inspect initial state, contacts, and randomization. |
| Teleoperation is delayed | Check Touch topics, newest-target handling, scaling, smoothing, and simulation rate. |
| Dataset looks wrong | Replay the affected episode; inspect frozen images, episode boundaries, action jumps, and gripper labels. |
| Checkpoint will not load | Match source version, task YAML, policy class, and Python dependencies. |
| Policy moves to the wrong pose | Check image order, normalization, action frame, quaternion convention, and reset distribution. |
| Real robot tracks poorly | Compare raw target with measured pose, then inspect latency, controller limits, and safety output. |

Change one variable at a time and repeat the smallest test that can confirm the cause. Do not collect a larger dataset until scene, teleoperation, recording, and deployment interfaces have been checked.

Chapter A

# Touch and OpenHaptics Installation

This appendix gives the complete device-specific installation used for the 3D Systems Touch. The validated stack is Ubuntu 22.04, ROS 2 Humble, OpenHaptics 3.4, and Touch Driver 2025.12.10. Install the general local environment from `README_ENV.md` first. Keep the Touch disconnected until the driver installation is complete.

## A.1 Download and extract the vendor packages

Download both Linux archives from the official 3D Systems support page:

https://support.3dsystems.com/s/article/OpenHaptics-for-Linux-Developer-Edition-v34?language=en_US

- **OpenHaptics Installer — OpenHaptics for Linux v3.4**;
- **Haptic Device Driver Installer — Touch Device Driver v2025.12.10**.

Create a permanent working directory and extract both archives. The browser may append a suffix such as `(1)` to a repeated download; adapt the archive name when necessary.

```
mkdir -p "$HOME/touch_driver"

tar -xzf "$HOME/Downloads/openhaptics_3.4-0-developer-edition-amd64.tar.gz" \
  -C "$HOME/touch_driver"
tar -xzf "$HOME/Downloads/TouchDriver_2025_12_10+1.tgz" \
  -C "$HOME/touch_driver"

ls "$HOME/touch_driver/TouchDriver_2025_12_10"
ls "$HOME/touch_driver/openhaptics_3.4-0-developer-edition-amd64"
```

Do not place these vendor archives inside the Git repository.

## A.2 Install dependencies and vendor software

Install the build, terminal, and OpenGL dependencies required by the SDK and its examples:

```
sudo apt update
sudo apt install -y \
  build-essential freeglut3-dev libglu1-mesa-dev \
  libncurses5-dev mesa-utils
```

Install the Touch driver first. This copies the device libraries and installs the USB permission rule:

```
cd "$HOME/touch_driver/TouchDriver_2025_12_10"
sudo bash ./install_haptic_driver
sudo udevadm control --reload-rules
sudo udevadm trigger
```

Reconnect the Touch after this command. Reboot if requested. Then install the OpenHaptics SDK:

```
cd "$HOME/touch_driver/openhaptics_3.4-0-developer-edition-amd64"
sudo ./install
sudo ldconfig
```

The SDK installer places examples and documentation under `/opt/OpenHaptics/Developer/3.4-0/`. The project also keeps the extracted SDK root so that CMake can locate the exact headers and libraries.

## A.3 Configure the SDK path

Define `OPENHAPTICS_ROOT` and verify the two files required by the project build:

```
export OPENHAPTICS_ROOT="$HOME/touch_driver/openhaptics_3.4-0-developer-edition-amd64"

test -f "$OPENHAPTICS_ROOT/usr/include/HD/hd.h" \
  && echo "OpenHaptics headers found"
test -f "$OPENHAPTICS_ROOT/usr/lib/libHD.so" \
  && echo "OpenHaptics HD library found"
```

Both messages must appear. Make the setting persistent for Bash terminals:

```
echo 'export OPENHAPTICS_ROOT="$HOME/touch_driver/openhaptics_3.4-0-developer-edition-amd64"' \
  >> "$HOME/.bashrc"
source "$HOME/.bashrc"
```

For Zsh, add the same export line to `~/.zshrc` instead.

Some vendor executables may report missing legacy terminal libraries such as `libtinfo.so.5` or `libncurses.so.5`. Install them only when that error occurs:

```
sudo apt install -y libtinfo5 libncurses5
```

## A.4 Validate the device before ROS

Plug in the Touch. First compile and run the read-only vendor diagnostic:

```
cd "$OPENHAPTICS_ROOT/opt/OpenHaptics/Developer/3.4-0/examples/HD/console/QueryDevice"
make
./QueryDevice
```

The program must identify the device, and the displayed pose and button values must change when the stylus moves. If initialization fails, stop here and fix the driver, USB connection, permissions, or missing library before using ROS.

Run the calibration example when the device requests calibration:

```
cd "$OPENHAPTICS_ROOT/opt/OpenHaptics/Developer/3.4-0/examples/HD/console/Calibration"
make
./Calibration
```

Finally, an optional force-feedback test verifies bidirectional communication:

```
cd "$OPENHAPTICS_ROOT/opt/OpenHaptics/Developer/3.4-0/examples/HD/console/FrictionlessSphere"
make
./FrictionlessSphere
```

Move the stylus gently; a virtual sphere should be perceptible. Keep a light grip and stop immediately with `Ctrl+C` if the device behaves unexpectedly.

## A.5 Build the project ROS 2 driver

Activate Conda, ROS 2, and the SDK path before building the workspace:

```
cd "$HOME/Stage_Lirmm/Diffusion-model-isaacsim"
conda activate microwave_dp
source /opt/ros/humble/setup.bash
export OPENHAPTICS_ROOT="$HOME/touch_driver/openhaptics_3.4-0-developer-edition-amd64"

cd ros2_WS
rosdep install --from-paths src --ignore-src -r -y
colcon build --symlink-install \
```

```
  --cmake-args -DOPENHAPTICS_ROOT="$OPENHAPTICS_ROOT"
source install/setup.bash
```

If the workspace was copied from another computer, old absolute paths may remain in generated files. Remove only the generated directories and rebuild:

```
cd "$HOME/Stage_Lirmm/Diffusion-model-isaacsim/ros2_WS"
rm -rf build install log
colcon build --symlink-install \
  --cmake-args -DOPENHAPTICS_ROOT="$OPENHAPTICS_ROOT"
source install/setup.bash
```

## A.6 Validate the ROS topics

Start the Touch node in terminal 1:

```
cd "$HOME/Stage_Lirmm/Diffusion-model-isaacsim"
conda activate microwave_dp
source /opt/ros/humble/setup.bash
source ros2_WS/install/setup.bash
ros2 launch touch_ros2_driver touch_rviz.launch.py
```

Inspect its outputs in terminal 2:

```
source /opt/ros/humble/setup.bash
source "$HOME/Stage_Lirmm/Diffusion-model-isaacsim/ros2_WS/install/setup.bash"
ros2 topic hz /touch/pose
ros2 topic echo /touch/buttons
```

The pose must update continuously and both buttons must change the published value. Stop both terminals with `Ctrl+C`. Do not continue to MuJoCo or a physical robot if the topic freezes, the axes jump while the device is still, or the node reports an OpenHaptics scheduler error.

## A.7 Daily use and common failures

In every new terminal used with the Touch, activate the three software layers:

```
cd "$HOME/Stage_Lirmm/Diffusion-model-isaacsim"
conda activate microwave_dp
source /opt/ros/humble/setup.bash
source ros2_WS/install/setup.bash
```

| Symptom | Check |
|---|---|
| Device not detected | Reconnect USB, reload the udev rules, reboot, and run `QueryDevice` before ROS. |
| Missing legacy terminal library | Install the two compatibility packages shown above. |
| CMake cannot find `HD/hd.h` | Correct `OPENHAPTICS_ROOT`, then delete only `ros2_WS/build`, `install`, and `log` before rebuilding. |
| Scheduler or device-in-use error | Close other OpenHaptics examples and ROS nodes; only one process should own the device. |
| ROS topics are absent | Source ROS 2 and `ros2_WS/install/setup.bash`, then inspect the first terminal for a driver error. |

Only after the vendor diagnostics and ROS topics pass should the Touch be linked to MuJoCo. Any later physical robot motion still requires the staged procedure described in section 4.4.